Tema 3 – Administración de Servidores Web

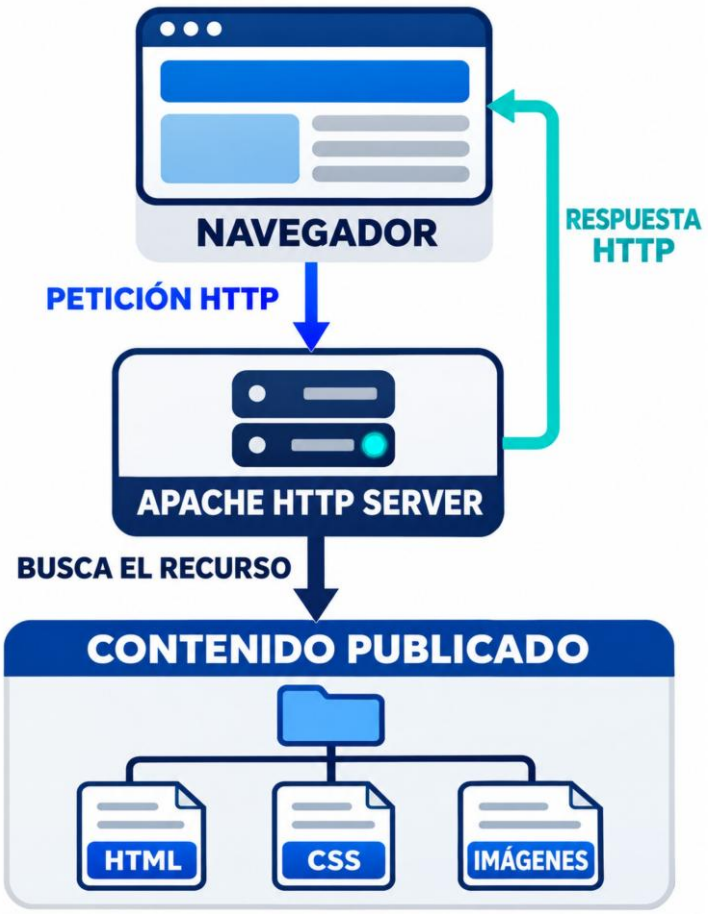


3.1 – Instalación y configuración básica de Apache HTTP Server

Función de Apache en una arquitectura web

Una arquitectura web necesita un componente capaz de recibir las peticiones enviadas por los navegadores y devolver los recursos solicitados. En este apartado utilizaremos Apache HTTP Server para asumir esa función. Apache permanecerá en ejecución en el servidor, escuchará las conexiones dirigidas al puerto configurado y decidirá qué respuesta debe ofrecer para cada petición.

Durante esta primera instalación, Apache servirá contenido estático: documentos HTML, hojas de estilo, imágenes y otros archivos almacenados en el sistema. Cuando el navegador solicite uno de esos recursos, el servidor localizará el archivo correspondiente, comprobará que puede leerlo y lo enviará dentro de una respuesta HTTP.



Este escenario es deliberadamente sencillo. Su finalidad es comprender y verificar el funcionamiento básico del servidor antes de incorporar sitios virtuales, autenticación, cifrado o integración con un servidor de aplicaciones. AulaDeploy todavía no se ejecutará como aplicación Java. Publicaremos una página de comprobación que permitirá confirmar que el servidor está preparado para las configuraciones posteriores.

Resultado esperado: Al finalizar el apartado, Apache estará instalado, activo y accesible desde otro equipo. También habrá una página estática publicada, un conjunto de pruebas realizadas y un registro de las operaciones efectuadas.

Servidor web, servicio Apache y sitio publicado

Para comprender con precisión el recorrido presentado, primero debemos diferenciar la máquina que actúa como servidor, el servicio Apache que atiende las peticiones y el sitio cuyos recursos se publican.

En una conversación técnica es frecuente utilizar «servidor» para referirse a elementos distintos. Para administrar correctamente Apache conviene diferenciar la máquina que proporciona los recursos, el programa que atiende las peticiones y el contenido que se publica. Los tres elementos colaboran, pero no son intercambiables.

Concepto	Significado	Ejemplo en el entorno de prácticas
Servidor	Equipo físico o virtual que dispone de sistema operativo, red, almacenamiento y capacidad de procesamiento.	La máquina virtual con Ubuntu Server.
Servicio web	Programa que permanece en ejecución y atiende peticiones mediante HTTP.	El servicio `apache2`.
Sitio web	Conjunto de recursos que el servidor web pone a disposición de los clientes.	La página de comprobación de AulaDeploy.

Instalar Apache significa incorporar al sistema los programas, archivos de configuración y utilidades necesarios. Iniciar Apache significa poner en ejecución el servicio. Publicar un sitio consiste en colocar sus recursos en una ubicación que Apache pueda servir y configurar correctamente su acceso.

Por tanto, que el paquete esté instalado no demuestra por sí solo que el servidor funcione. El servicio podría estar detenido, escuchar en un puerto inesperado, no tener acceso a los archivos o quedar bloqueado por la red. La comprobación completa debe revisar cada una de esas capas.

Recorrido de una petición hasta un archivo estático

Una vez diferenciados el servidor, el servicio y el sitio publicado, podemos seguir una petición concreta desde que el usuario escribe la URL hasta que Apache devuelve el archivo solicitado.



La ruta solicitada se relaciona con un archivo situado bajo el DocumentRoot.

La dirección utilizada por el navegador identifica el equipo, el puerto y la ruta solicitada. En una petición dirigida a `http://192.168.56.20/estado.html`, el navegador intenta establecer una conexión con el puerto 80 del servidor y solicita el recurso `/estado.html`.

Apache relaciona esa ruta con el directorio desde el que publica el sitio. En la configuración inicial de Ubuntu, ese directorio es `/var/www/html`. Por ello, la petición de `/estado.html` se corresponde con el archivo `/var/www/html/estado.html`. Si el archivo existe, Apache puede leerlo y la configuración permite servirlo, la respuesta habitual será `200 OK`.

Esta correspondencia no significa que el navegador pueda recorrer libremente el sistema de archivos. Apache solo publica los recursos comprendidos dentro de los espacios autorizados por su configuración. Una ruta de la URL y una ruta del sistema de archivos representan realidades diferentes que el servidor relaciona de forma controlada.

Entorno común de prácticas

El recorrido anterior solo podrá reproducirse si el servidor dispone de un entorno conocido y de una conexión de red correctamente identificada. Por ello, antes de instalar Apache prepararemos y documentaremos la máquina virtual de prácticas.

Todas las prácticas se realizarán sobre una máquina virtual común para evitar que las diferencias entre equipos consuman el tiempo destinado al aprendizaje. La máquina utilizará Ubuntu Server 24.04 LTS y dispondrá de una interfaz de red que permita la comunicación con el equipo anfitrión.

Antes de instalar Apache debe registrarse la información básica del entorno. Estos datos permitirán interpretar las órdenes posteriores, distinguir la dirección del servidor de la dirección del cliente y repetir las pruebas en condiciones conocidas.

Dato	Valor que debe anotarse
Nombre de la máquina virtual	
Versión del sistema operativo	
Nombre del usuario de trabajo	
Dirección IPv4 del servidor	
Nombre de la interfaz de red	
Dirección IPv4 del equipo anfitrión	
Tipo de red de la máquina virtual	
Fecha de la práctica	

La dirección mostrada en los ejemplos, `192.168.56.20`, es únicamente ilustrativa. En todas las pruebas debe sustituirse por la dirección real obtenida en la máquina virtual.

Comprobaciones previas del sistema

Con el entorno de prácticas identificado, el siguiente paso consiste en comprobar que el sistema reúne las condiciones mínimas para realizar la instalación sin atribuir a Apache problemas que pertenecen al sistema o a la red.

Antes de modificar el sistema conviene verificar su identidad, la sesión utilizada, el espacio disponible y la configuración de red. Estas comprobaciones permiten detectar errores que no dependen de Apache, como trabajar en la máquina equivocada, carecer de privilegios administrativos o no disponer de una dirección de red válida.

Órdenes

Identificar el sistema operativo

```
cat /etc/os-release
```

Identificar al usuario

```
whoami
```

Comprobar los privilegios

```
sudo -v
```

Revisar el espacio disponible

```
df -h /
```

Consultar las direcciones de red

```
ip -brief address
```

Qué debe comprobarse

Sistema operativo

La salida debe identificar Ubuntu y mostrar su versión. Los nombres de paquetes y la organización de Apache que emplearemos corresponden a la familia Debian y Ubuntu.

Usuario de sesión

La orden muestra el usuario de la sesión. No es necesario iniciar sesión directamente como root; utilizaremos `sudo` cuando una operación requiera privilegios administrativos.

Privilegios administrativos

Si la contraseña es correcta y el usuario dispone de permisos, la orden valida temporalmente sus credenciales administrativas.

Espacio en disco

Debe existir espacio suficiente en la partición raíz.

Interfaces de red

Debe aparecer al menos una interfaz activa con una dirección distinta de `127.0.0.1`.

Criterio para continuar: La instalación puede comenzar cuando el sistema está correctamente identificado, `sudo` funciona, existe espacio disponible y la máquina dispone de una dirección de red utilizable.

Actualización del índice de paquetes

Superadas las comprobaciones del sistema, ya podemos preparar el gestor de paquetes para localizar una versión adecuada de Apache en los repositorios configurados.

Ubuntu instala programas mediante paquetes obtenidos de repositorios configurados. Antes de solicitar un paquete nuevo, el sistema necesita consultar qué versiones están disponibles y de dónde debe descargarlas. Esta información se guarda en un índice local.

Ejecuta:

```
sudo apt update
```

La orden descarga información actualizada de los repositorios, pero no sustituye automáticamente todos los programas instalados. Su finalidad en esta práctica es asegurar que `apt` conoce la versión de Apache disponible para el sistema.

Operación	Efecto
`sudo apt update`	Actualiza el índice local de paquetes disponibles.
`sudo apt upgrade`	Actualiza paquetes ya instalados cuando existen versiones nuevas.

En esta práctica solo necesitamos ejecutar `apt update`. Si aparecen errores de resolución de nombres, conexión o repositorios inaccesibles, no debe continuarse como si la operación hubiera terminado correctamente. Primero hay que comprobar la configuración de red y la disponibilidad del repositorio indicado en el mensaje.

Resultado esperado: La orden debe finalizar sin errores que impidan utilizar los repositorios. Los avisos deben leerse, pero no todos los avisos significan que la operación haya fallado.

Instalación de Apache HTTP Server

Con el índice de paquetes actualizado, el sistema ya dispone de la información necesaria para resolver las dependencias, descargar e instalar Apache HTTP Server.

En Ubuntu, Apache HTTP Server se distribuye mediante el paquete `apache2`. Al instalarlo, el gestor de paquetes incorpora el servidor, sus utilidades, los archivos de configuración iniciales y las dependencias necesarias.

Ejecuta:

```
sudo apt install apache2
```

Cuando el sistema solicite confirmación, revisa el resumen de la operación y responde afirmativamente. La descarga y el tamaño pueden variar según los paquetes que ya estén presentes en la máquina.

La instalación mediante `apt` aporta varias ventajas: utiliza paquetes preparados para la distribución, mantiene una estructura coherente de directorios, integra Apache con el gestor de servicios y permite recibir actualizaciones mediante los mecanismos habituales del sistema. No estamos compilando Apache desde su código fuente ni eligiendo manualmente cada componente.

Al terminar, Ubuntu suele iniciar el servicio automáticamente. Sin embargo, no debe darse por hecho que todo funciona. El proceso de validación comienza después de instalar el paquete.



Resultado esperado: La orden debe finalizar sin mensajes de error y devolver el control a la consola. La mera finalización del instalador confirma que el paquete se ha procesado, pero todavía debemos comprobar la versión, el estado del servicio, el puerto y la respuesta HTTP.

Comprobación del paquete y de la versión instalada

La instalación devuelve el control a la consola cuando termina, pero ese resultado no basta para documentar qué se ha instalado realmente. Debemos identificar el paquete y su versión.

Después de la instalación debemos registrar qué paquete y qué versión se han incorporado realmente. Esta comprobación evita redactar la documentación a partir de una versión supuesta.

Paquete instalado

Comprobar el paquete

```
dpkg -l apache2
```

La línea correspondiente a `apache2` debe comenzar con el estado `ii`, que indica que el paquete está seleccionado e instalado.

También puede consultarse la versión disponible y la versión instalada mediante:

```
apt-cache policy apache2
```

Versión de Apache

Comprobar la versión del servidor

```
apache2ctl -v
```

La salida mostrará la versión de Apache y la fecha de compilación. El número concreto puede cambiar cuando Ubuntu publique actualizaciones, por lo que debe anotarse el resultado real y no copiarse un número fijo del material.

Obtener información adicional

```
apache2ctl -V
```

Esta variante muestra datos de compilación, rutas y parámetros generales. No es necesario interpretar todavía toda la salida.

Estado del paquete

Resultado de `dpkg -l apache2`

Versión instalada

Resultado de `apache2ctl -v`

Fecha de la comprobación

Registrar la fecha real

Orden utilizada

Anotar la orden exacta

Advertencias relevantes

Cualquier aviso que aparezca

Estado del servicio Apache

Después de confirmar la presencia del paquete, debemos comprobar si el servicio que proporciona está realmente en ejecución y preparado para atender peticiones.

El paquete instalado proporciona el programa, pero las peticiones solo podrán atenderse si el servicio está en ejecución. Ubuntu administra Apache mediante systemd, que registra su estado y permite iniciar, detener o recargar el servicio de manera controlada.

Ejecuta:

```
sudo systemctl status apache2 --no-pager
```

La salida debe mostrar información similar a la siguiente:

```
Loaded: loaded (...; enabled; ...)
Active: active (running)
```

`Loaded` indica que systemd reconoce la unidad que define el servicio. La palabra `enabled` señala que está configurado para iniciarse automáticamente. `Active: active (running)` confirma que el servicio se encuentra en ejecución.

Para obtener una respuesta más breve puede utilizarse:

```
systemctl is-active apache2
```

El resultado esperado es:

```
active
```

Si el servicio aparece como `failed`, no debe intentarse reinstalar Apache inmediatamente. Primero deben consultarse los mensajes incluidos en `systemctl status` y el registro de errores, porque suelen indicar la causa del problema.

Paquete instalado

El programa existe en el sistema, pero no implica que esté en ejecución.

Servicio activo

El servicio está en ejecución y puede atender peticiones HTTP. Ambas comprobaciones deben documentarse.

Inicio, detención, reinicio y recarga de Apache

Una vez conocido el estado del servicio, podemos practicar las operaciones básicas que permiten controlarlo de forma deliberada y comprobar el resultado de cada acción.

La administración cotidiana de Apache exige controlar el estado del servicio. Cada operación responde a una necesidad distinta.

Operación	Orden	Efecto
Iniciar	`sudo systemctl start apache2`	Pone en ejecución un servicio detenido.
Detener	`sudo systemctl stop apache2`	Finaliza el servicio y deja de atender peticiones.
Reiniciar	`sudo systemctl restart apache2`	Detiene el servicio y lo inicia de nuevo.
Recargar	`sudo systemctl reload apache2`	Vuelve a leer la configuración procurando mantener las conexiones existentes.

Comprobación guiada

Detén el servicio:

```
sudo systemctl stop apache2
systemctl is-active apache2
```

El resultado esperado es `inactive`. A continuación, vuelve a iniciarlo:

```
sudo systemctl start apache2
systemctl is-active apache2
```

El resultado esperado es `active`.

La recarga suele ser preferible cuando solo se han realizado cambios de configuración compatibles con esta operación. El reinicio implica una interrupción mayor y debe utilizarse cuando la modificación o el estado del servicio lo requieran.

⚠ No ejecutes varias órdenes diferentes de forma mecánica. Después de cada operación, comprueba si el estado obtenido coincide con el objetivo perseguido.

Inicio automático y procesos de Apache

El control manual del servicio resuelve las operaciones inmediatas, pero un servidor también debe recuperarse después de reiniciar el sistema y ejecutar sus procesos con una separación adecuada de privilegios.

Inicio automático

Un servidor destinado a prestar servicio debe recuperar su funcionamiento después de reiniciar el sistema. Para comprobar si Apache está configurado para iniciarse automáticamente, ejecuta:

```
systemctl is-enabled apache2
```

El resultado esperado es:

```
enabled
```

Si el servicio no estuviera habilitado, podría activarse mediante:

```
sudo systemctl enable apache2
```

Habilitar un servicio y ponerlo en ejecución son operaciones relacionadas, pero diferentes. `enable` configura su inicio automático; `start` lo inicia en la sesión actual.

Procesos de Apache

Apache no atiende necesariamente todas las peticiones mediante un único proceso. La configuración utilizada en Ubuntu mantiene un proceso principal y crea procesos o hilos que participan en la atención de las conexiones.

Para observar los procesos existentes, ejecuta:

```
ps -C apache2 -o pid,user,group,cmd
```



La implementación concreta depende del módulo de multiprocesamiento activo.

Direcciones de escucha y puerto 80

Aunque Apache aparezca activo, todavía debemos comprobar que está escuchando en el punto de red esperado, porque un servicio en ejecución no garantiza por sí solo que acepte conexiones en el puerto 80.

Para recibir una conexión, un servicio debe escuchar en una combinación de dirección y puerto. HTTP utiliza habitualmente el puerto TCP 80. Una vez instalado Apache, debemos comprobar que existe un proceso escuchando en ese puerto.

Interfaz de red

eth0 / enp0s3 (ejemplo)

Puerto TCP 80

Estado: LISTEN

Servicio Apache

`apache2` en ejecución

Ejecuta:

```
sudo ss -ltnp 'sport = :80'
```

El filtro `sport = :80` limita la consulta a los sockets cuyo puerto local es exactamente el 80. De este modo se evita confundir el resultado con otros puertos que también comiencen por 80, como 8000 u 8080.

`0.0.0.0:80` indica que el servicio acepta conexiones IPv4 dirigidas al puerto 80 a través de las interfaces disponibles. `[::]:80` representa la escucha mediante IPv6 y, dependiendo de la configuración del sistema, también puede atender conexiones compatibles.

El puerto utilizado por Apache se declara en:

```
/etc/apache2/ports.conf
```

Puede consultarse sin modificarlo mediante:

```
cat /etc/apache2/ports.conf
```

En este apartado mantendremos el puerto 80. La publicación mediante otros puertos y la creación de sitios diferenciados se estudiarán posteriormente.

Comprobación necesaria: Un servicio puede aparecer activo y, sin embargo, no escuchar donde esperamos. Por eso deben comprobarse tanto el estado de systemd como el socket de red.

Comprobación local mediante navegador y curl

Confirmada la escucha en el puerto 80, realizaremos primero una petición desde el propio servidor para validar Apache sin introducir todavía posibles problemas de la red externa.

La primera petición debe realizarse desde el propio servidor. Esta prueba reduce el número de elementos implicados: si funciona localmente, sabemos que Apache responde antes de analizar el acceso desde otros equipos.

Prueba mediante navegador

Si la máquina dispone de entorno gráfico, abre:

```
http://localhost
```

También puede utilizarse:

```
http://127.0.0.1
```

Prueba mediante consola

En Ubuntu Server es habitual no disponer de navegador gráfico. Utiliza:

```
curl http://localhost
```

La orden muestra el cuerpo de la respuesta, que inicialmente corresponderá a la página predeterminada de Apache. Para consultar solo las cabeceras:

```
curl -I http://localhost
```

La primera línea debería contener un estado satisfactorio, normalmente:

```
HTTP/1.1 200 OK
```

200 OK

La petición se ha procesado correctamente y el servidor devuelve el recurso solicitado.

Cabeceras adicionales

Aportan información sobre el tipo de contenido, su tamaño o la fecha de la respuesta.

Si la prueba local falla

Revisar primero el servicio, el puerto y los registros de Apache. No buscar el problema en la red externa.

Acceso a Apache desde otro equipo

La respuesta local demuestra que Apache funciona dentro del servidor. La siguiente prueba amplía el recorrido y comprueba si otro equipo puede alcanzar el servicio a través de la red virtual.



La prueba local confirma que Apache responde dentro del servidor, pero no demuestra que otros equipos puedan alcanzarlo. La siguiente comprobación debe realizarse desde el equipo anfitrión o desde otra máquina conectada a la misma red.

1 Obtener la dirección del servidor

En Ubuntu ejecuta `hostname -I`. Identifica la dirección correspondiente a la interfaz utilizada en la práctica. No emplees `127.0.0.1`, porque esa dirección siempre se refiere al propio equipo desde el que se realiza la conexión.

2 Comprobar la conectividad

Desde el equipo cliente, comprueba que puede alcanzar al servidor. La herramienta utilizada dependerá del sistema operativo y de la configuración de la red.

3 Abrir el sitio

En el navegador del equipo cliente escribe: `http://DIRECCIÓN_IP_DEL_SERVIDOR`. Ejemplo: `http://192.168.56.20`

4 Interpretar el resultado

Si aparece la página de Apache, la red, el puerto y el servicio permiten el acceso. Si la prueba local funcionaba pero la remota no, deben revisarse la dirección utilizada, el modo de red de la máquina virtual y el cortafuegos.

Evidencia: Conserva una captura en la que puedan identificarse la dirección utilizada y la página recibida.

Cortafuegos UFW y acceso HTTP

Si el acceso local funciona pero el acceso desde otro equipo falla, uno de los elementos que debemos revisar es el cortafuegos del servidor.

Un cortafuegos local puede impedir el acceso aunque Apache esté activo y escuche en el puerto correcto. Ubuntu proporciona UFW como interfaz para administrar reglas sencillas. No debe modificarse el cortafuegos sin comprobar antes su estado.

Ejecuta:

```
sudo ufw status verbose
```

Si UFW está inactivo

La salida indicará:

```
Status: inactive
```

En ese caso, UFW no está bloqueando la conexión. No es necesario activarlo únicamente para completar esta práctica.

Si UFW está activo

Consulta los perfiles disponibles:

```
sudo ufw app list
```

Comprueba qué permite el perfil de Apache:

```
sudo ufw app info Apache
```

Autoriza HTTP:

```
sudo ufw allow Apache
```

Comprueba el resultado:

```
sudo ufw status
```

El perfil Apache permite normalmente el tráfico HTTP por el puerto 80. HTTPS se configurará en un apartado posterior y no debe habilitarse anticipadamente mediante una configuración que todavía no se ha implantado.

⚠ En una administración remota, modificar el cortafuegos sin conservar el acceso SSH puede dejar el servidor inaccesible. Las reglas deben aplicarse de forma razonada y documentada.

Estructura de directorios de Apache en Ubuntu

Una vez validado el acceso al servicio, podemos pasar de observar Apache desde el exterior a conocer cómo organiza internamente su configuración, su contenido y sus registros.

Árbol de directorios



Para observar la estructura ejecuta:

```
ls -l /etc/apache2
```

También puedes listar los directorios de los dos primeros niveles:

```
sudo find /etc/apache2 -maxdepth 2 -type d | sort
```

Explicación de cada elemento

`/etc/apache2/` contiene la configuración del servidor. Los directorios terminados en `-available` almacenan elementos disponibles, mientras que los terminados en `-enabled` contienen enlaces a los elementos activados.

`/var/www/html/` es el directorio de publicación utilizado por el sitio predeterminado. Los recursos depositados aquí pueden relacionarse con las rutas solicitadas por el navegador.

`/var/log/apache2/` contiene los registros principales. En ellos quedan anotadas las peticiones recibidas y los errores detectados.

La estructura concreta pertenece al empaquetado de Debian y Ubuntu; otras distribuciones pueden utilizar nombres y rutas diferentes.

Archivos principales de configuración

El mapa de directorios anterior permite localizar los tres archivos que gobiernan los aspectos más generales del servidor y comprender qué función cumple cada uno.

Apache obtiene su comportamiento a partir de archivos de texto que contienen directivas. En Ubuntu, la configuración principal se encuentra en `/etc/apache2/`.

/etc/apache2/apache2.conf

Es el archivo principal. Define opciones globales e incorpora otros archivos y directorios mediante directivas de inclusión. No conviene modificarlo sin comprender el alcance global del cambio.

Puede consultarse mediante:

```
sudo less /etc/apache2/apache2.conf
```

Pulsa `q` para salir de `less`.

/etc/apache2/ports.conf

Declara los puertos en los que Apache debe escuchar. En la configuración inicial encontraremos la escucha asociada al puerto 80.

```
cat /etc/apache2/ports.conf
```

/etc/apache2/envvars

Define variables utilizadas durante el arranque, como el usuario, el grupo y determinados directorios internos.

```
cat /etc/apache2/envvars
```

La configuración efectiva de Apache no procede únicamente de estos tres archivos. El archivo principal incluye configuraciones disponibles y activadas para módulos, sitios y opciones adicionales. Esta separación permite administrar el servidor de manera ordenada.

⚠ **Importante:** No modifiques archivos para «probar qué ocurre» sin crear una copia, saber qué directiva estás cambiando y validar después la sintaxis.

Configuración del sitio predeterminado

Conocidos los archivos globales, debemos observar cómo utiliza Apache esa configuración para publicar el sitio predeterminado instalado por Ubuntu.

Ubuntu instala un sitio predeterminado cuya configuración se encuentra en:

```
/etc/apache2/sites-available/000-default.conf
```

Puede consultarse mediante:

```
sudo sed -n '1,160p' /etc/apache2/sites-available/000-default.conf
```

Fragmento de configuración

```
<VirtualHost *:80>
    ServerAdmin webmaster@localhost
    DocumentRoot /var/www/html
    ErrorLog ${APACHE_LOG_DIR}/error.log
    CustomLog
${APACHE_LOG_DIR}/access.log combined
</VirtualHost>
```

Explicación de cada directiva

`<VirtualHost *:80>` delimita una configuración que atiende en el puerto 80.

`ServerAdmin` establece una dirección de contacto administrativa.

`DocumentRoot` indica el directorio que contiene los recursos publicados.

`ErrorLog` y `CustomLog` determinan los archivos utilizados para registrar errores y accesos.

En este apartado utilizaremos el sitio predeterminado sin crear otros nuevos. La existencia de un bloque `VirtualHost` no implica que debamos estudiar todavía las diferentes formas de alojar varios sitios. La creación de sitios virtuales por nombre, dirección IP o puerto pertenece al apartado siguiente.

DocumentRoot y correspondencia entre URL y archivo

Dentro de la configuración del sitio predeterminado, DocumentRoot establece la relación fundamental entre una ruta solicitada por el navegador y un archivo almacenado en el servidor.

La directiva `DocumentRoot` establece el punto del sistema de archivos desde el que Apache publica el sitio. En el sitio predeterminado de Ubuntu su valor es `/var/www/html`.



Cuando el navegador solicita la raíz `/`, Apache busca un documento de índice, normalmente `index.html`. Para una ruta como `/assets/css/estilos.css`, el servidor busca el recurso dentro de la estructura situada bajo el `DocumentRoot`.

Esta relación permite organizar el sitio mediante subdirectorios. Sin embargo, no debe confundirse con una autorización general para acceder al sistema. Los archivos situados fuera de los espacios permitidos no quedan publicados simplemente porque el usuario escriba una ruta en la URL.

Antes de copiar contenido debe comprobarse que el archivo se encuentra bajo el directorio correcto, que Apache dispone de permiso de lectura, que los directorios del recorrido permiten el acceso, que la URL mantiene la misma estructura relativa y que el nombre respeta mayúsculas y minúsculas.

En GNU/Linux, `Estilos.css` y `estilos.css` son nombres diferentes.

Sustitución segura de la página predeterminada

Una vez comprendida esa correspondencia, podemos sustituir la página inicial de Apache sin perder el archivo original ni modificar otros espacios del sistema.

La página inicial de Apache demuestra que la instalación funciona, pero no representa el sitio que queremos publicar. Antes de sustituirla, conservaremos una copia fuera del directorio público.

1 Comprobar el archivo existente

```
ls -l /var/www/html/index.html
```

2 Crear una copia de seguridad

```
sudo cp /var/www/html/index.html /var/backups/apache2-index.html
```

La copia se guarda en `/var/backups`, fuera del directorio publicado.

3 Comprobar la copia

```
ls -l /var/backups/apache2-index.html
```

4 Preparar la nueva estructura

```
sudo mkdir -p /var/www/html/assets/css
```

La opción `-p` crea los directorios necesarios y no produce error si ya existen.

No es necesario detener Apache para sustituir archivos estáticos. El servidor leerá el contenido solicitado cuando reciba una nueva petición. Tampoco es necesario reiniciarlo por crear o modificar un documento HTML o CSS.

Idea clave: Los cambios en los archivos publicados y los cambios en la configuración del servidor son operaciones distintas. Los primeros suelen verse en la siguiente petición; los segundos pueden requerir una recarga o un reinicio.

Estructura de la página de comprobación de AulaDeploy

Después de conservar la página predeterminada, prepararemos la estructura mínima del contenido que servirá para comprobar conjuntamente la publicación de HTML y CSS.

Crearemos una página estática que servirá como evidencia de que Apache puede publicar correctamente HTML y CSS. Esta página no es todavía la aplicación AulaDeploy ni sustituye al archivo WAR que se desplegará posteriormente en Tomcat.

Árbol de archivos



`index.html` contendrá la estructura y el contenido. `assets/css/estilos.css` contendrá la presentación. La ruta utilizada en el atributo `href` será relativa al documento HTML, por lo que el navegador solicitará `/assets/css/estilos.css` al mismo servidor.

El diseño permitirá comprobar dos recursos diferentes: si aparece el contenido sin estilos, el HTML se ha recibido, pero la hoja CSS no. Si no aparece ningún contenido, debe investigarse primero el acceso al documento principal.

Código HTML de la página de comprobación

Definida la estructura de archivos, comenzaremos creando el documento HTML que proporcionará el contenido y solicitará la hoja de estilo mediante una ruta relativa.

Abre el archivo:

```
sudo nano /var/www/html/index.html
```

Sustituye su contenido por:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>AulaDeploy | Apache operativo</title>
  <link rel="stylesheet" href="assets/css/estilos.css">
</head>
<body>
  <header class="cabecera">
    <p class="etiqueta">ENTORNO DE PRÁCTICAS</p>
    <h1>AulaDeploy</h1>
    <p>Apache HTTP Server operativo</p>
  </header>
  <main class="contenido">
    <section>
      <h2>Servicio</h2>
      <p>Apache está instalado y atiende peticiones HTTP.</p>
    </section>
    <section>
      <h2>Publicación</h2>
      <p>El contenido se sirve desde el directorio configurado.</p>
    </section>
    <section>
      <h2>Siguiente etapa</h2>
      <p>La configuración se ampliará de forma progresiva.</p>
    </section>
  </main>
  <footer>DAW · Despliegue de aplicaciones web</footer>
</body>
</html>
```

Guarda con `Ctrl+O`, confirma con `Intro` y sal con `Ctrl+X`.

Elementos principales

El enlace a la hoja de estilo utiliza una ruta relativa. Los tres elementos `section` permiten comprobar que el navegador interpreta correctamente la estructura del documento.

> **`<link>`**
>
> Ruta relativa a la hoja de estilo CSS.

> **`<header>`**
>
> Cabecera con nombre e indicación de estado.

> **`<section> ×3`**
>
> Tres bloques de comprobación independientes.

> **`<footer>`**
>
> Identificación del entorno de prácticas.

Hoja de estilo de la página de comprobación

Con el documento HTML creado, añadiremos la hoja CSS solicitada por el navegador para comprobar que Apache puede servir correctamente recursos situados en subdirectorios.

Abre la hoja de estilo:

```
sudo nano /var/www/html/assets/css/estilos.css
```

Introduce:

```
:root {
  color-scheme: light;
  --azul: #173b67;
  --cian: #19a7ce;
  --fondo: #f3f7fb;
  --texto: #1f2937;
}
* { box-sizing: border-box; }
body {
  margin: 0;
  min-height: 100vh;
  font-family: Arial, sans-serif;
  color: var(--texto);
  background: var(--fondo);
}
.cabecera {
  padding: 4rem 8vw;
  color: white;
  background: linear-gradient(135deg, var(--azul), #245b94);
}
.etiqueta {
  color: #a8edff;
  font-weight: bold;
  letter-spacing: 0.12em;
}
.cabecera h1 {
  margin: 0.3rem 0;
  font-size: 3rem;
}
.contenido {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 1.5rem;
  padding: 3rem 8vw;
}
.contenido section {
  padding: 1.5rem;
  background: white;
  border-top: 4px solid var(--cian);
  box-shadow: 0 0.5rem 1.5rem #173b6715;
}
footer {
  padding: 1.5rem 8vw;
  color: white;
  background: #102a49;
}
```

Guarda y cierra el archivo. No es necesario reiniciar Apache porque solo se ha modificado contenido estático.

Propósito del CSS

La hoja de estilo se utiliza únicamente para comprobar la publicación de varios recursos. Si la página aparece con diseño aplicado, confirma que Apache ha servido correctamente tanto el documento HTML como el archivo CSS.

Propietarios y permisos del contenido publicado

La existencia de los archivos no garantiza que Apache pueda leerlos. Antes de validar la publicación debemos revisar sus propietarios y conceder únicamente los permisos necesarios.

Apache necesita leer los archivos y atravesar los directorios que forman la ruta hasta ellos. No necesita permiso de escritura sobre el contenido estático para publicarlo. Conceder más permisos de los necesarios aumenta el riesgo de modificaciones no autorizadas.

Aplicaremos una configuración sencilla para el entorno de prácticas:

```
sudo chown -R root:www-data /var/www/html
sudo find /var/www/html -type d -exec chmod 755 {} \;
sudo find /var/www/html -type f -exec chmod 644 {} \;
```

Elemento	Permiso	Interpretación
Directorios	755	El propietario puede modificar; los demás pueden recorrer y leer.
Archivos	644	El propietario puede modificar; los demás pueden leer.

Comprueba el resultado:

```
ls -l /var/www/html
ls -l /var/www/html/assets/css
```

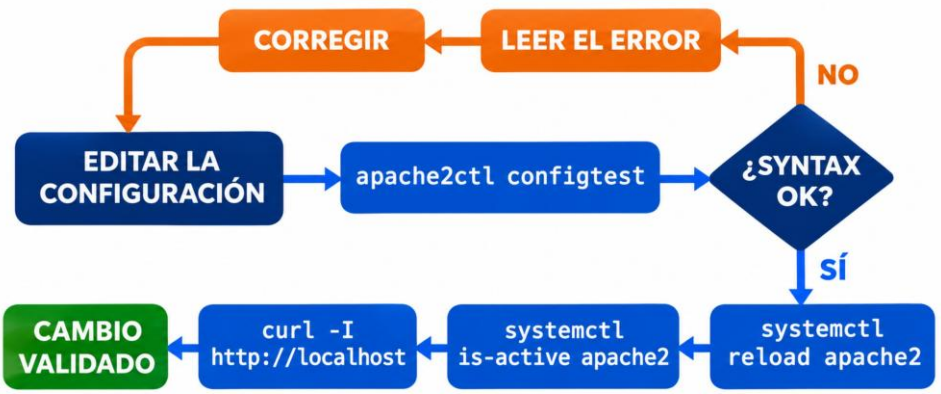
El propietario será `root`, el grupo será `www-data` y los archivos podrán leerse sin conceder escritura general.

⚠ **Advertencia:** No utilices `chmod 777` para resolver un problema de permisos. Esa orden concede escritura a todos los usuarios y oculta la causa real en lugar de corregirla. Los permisos deben responder a las operaciones que necesita cada identidad. En este entorno, la edición posterior requerirá `sudo`, lo que deja claro cuándo se modifica contenido administrado.

Validación y aplicación segura de cambios

Después de publicar el contenido y ajustar sus permisos, aplicaremos un procedimiento seguro para validar cualquier cambio de configuración antes de recargar el servicio.

Antes de aplicar un cambio de configuración debe comprobarse su sintaxis. Apache proporciona una utilidad que analiza los archivos cargados y avisa de determinados errores.



Validar antes de recargar evita aplicar una configuración con errores de sintaxis.

Ejecuta:

```
sudo apache2ctl configtest
```

El resultado esperado es:

```
Syntax OK
```

Si aparece un error, debe leerse el archivo y el número de línea indicados. No recargues el servicio hasta corregirlo y obtener `Syntax OK`.

Cuando la validación sea correcta, aplica los cambios:

```
sudo systemctl reload apache2
```

Comprueba el estado:

```
systemctl is-active apache2
```

Finalmente, realiza una petición:

```
curl -I http://localhost
```

Una configuración sintácticamente correcta todavía puede contener un valor inadecuado. Por ello, `Syntax OK` es necesario, pero no sustituye a la prueba funcional.

Registros de acceso y error de Apache

La validación de sintaxis y la prueba HTTP indican si el cambio funciona, pero los registros permiten reconstruir lo sucedido sin depender únicamente de lo que muestra el navegador. En la configuración predeterminada de Ubuntu se encuentran en `/var/log/apache2/`.

Registro de acceso

```
/var/log/apache2/access.log
```

Contiene información sobre las peticiones recibidas: dirección del cliente, fecha, recurso solicitado, estado devuelto y tamaño de la respuesta.

Consulta las últimas entradas:

```
sudo tail -n 20 /var/log/apache2/access.log
```

Registro de error

```
/var/log/apache2/error.log
```

Recoge problemas detectados por Apache, mensajes de inicio y determinadas incidencias relacionadas con la configuración o los recursos.

Consulta las últimas entradas:

```
sudo tail -n 30 /var/log/apache2/error.log
```

Para observar nuevas entradas en tiempo real:

```
sudo tail -f /var/log/apache2/access.log
```

Realiza una petición desde el navegador y comprueba que aparece una nueva línea. Finaliza el seguimiento con `Ctrl+C`.



`access.log` permite comprobar qué petición ha llegado y qué estado se ha devuelto. `error.log` ayuda a investigar fallos de arranque, configuración o acceso a los recursos.

Relación entre peticiones, estados y registros

Una vez localizados los registros, relacionaremos cada petición con el código de estado devuelto para distinguir un funcionamiento correcto de un recurso inexistente o un acceso denegado.

Una prueba es más útil cuando se relaciona lo que solicita el cliente con lo que registra el servidor. Los códigos de estado permiten clasificar el resultado, pero deben interpretarse junto con la petición realizada y los mensajes disponibles.

Estado	Significado en esta práctica	Posible comprobación
`200`	El recurso se ha servido correctamente.	Solicitar `/` o un archivo existente.
`304`	El cliente ha realizado una petición condicional y el recurso no ha cambiado, por lo que puede reutilizar la copia almacenada.	Recargar desde un navegador y comprobar la petición en sus herramientas de red, si se ha enviado una validación condicional.
`403`	Apache ha entendido la petición, pero deniega el acceso.	Revisar permisos y autorización.
`404`	El recurso solicitado no se ha encontrado.	Revisar la URL y el nombre del archivo.
`500`	Se ha producido un error interno.	Consultar inmediatamente `error.log`.

La petición de un recurso inexistente debe aparecer en access.log con el estado 404. Una respuesta 404 ordinaria no tiene por qué generar una entrada específica en error.log; este registro se consulta para comprobar si Apache ha anotado información adicional relacionada.

Práctica guiada

Genera una petición correcta:

```
curl -I http://localhost/
```

Genera una petición a un recurso inexistente:

```
curl -I http://localhost/no-existe.html
```

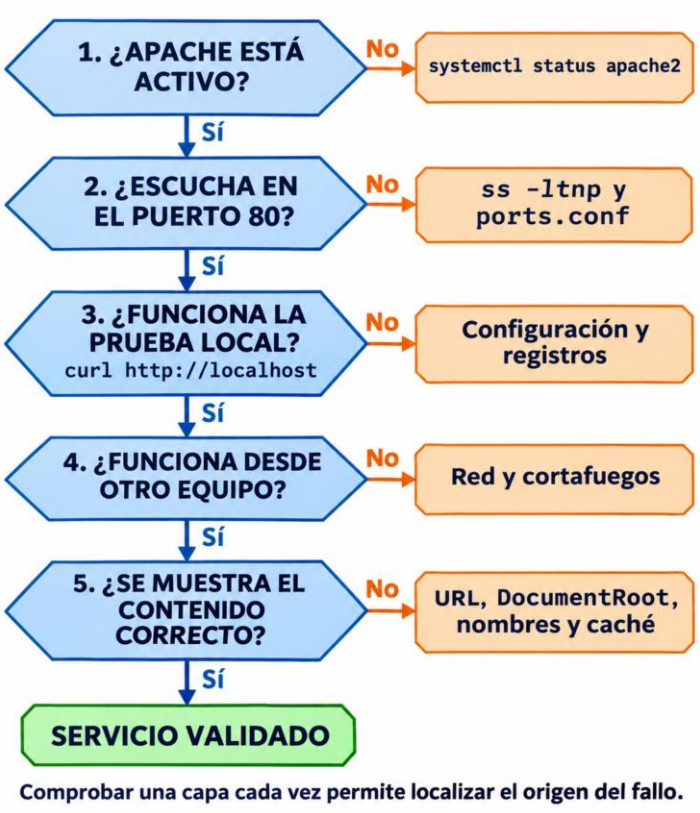
Consulta después:

```
sudo tail -n 10 /var/log/apache2/access.log
sudo tail -n 20 /var/log/apache2/error.log
```

Identifica en `access.log` las dos rutas y sus códigos. Una entrada `404` demuestra que Apache sí ha recibido y procesado la petición; el problema es que no ha encontrado el recurso indicado.

Diagnóstico sistemático de incidencias

Los códigos de estado y los registros aportan evidencias, pero deben consultarse siguiendo un orden para evitar modificar componentes que ya funcionan correctamente.



Comprobar una capa cada vez permite localizar el origen del fallo.

Tabla de incidencias

Síntoma	Comprobación inicial
Conexión rechazada	Estado del servicio y puerto de escucha.
Tiempo de espera agotado	Dirección, red virtual, ruta y cortafuegos.
403 Forbidden	Permisos y reglas de acceso.
404 Not Found	URL, nombre y ubicación del archivo.
Página antigua	Caché, archivo modificado y servidor consultado.
Apache no recarga	`apache2ctl configtest` y `error.log`.
HTML sin estilos	Ruta de la hoja CSS y petición registrada.

El diagnóstico debe avanzar desde las capas más básicas hacia el contenido. Cambiar varias configuraciones a la vez impide saber qué modificación ha resuelto o agravado el problema.

Plan de validación del servidor web

El procedimiento de diagnóstico puede convertirse ahora en un plan de validación que reúna todas las comprobaciones realizadas desde la instalación hasta la consulta de los registros.

La validación final no consiste en abrir una vez el navegador. Debe reunir pruebas diferentes que confirmen la instalación, la ejecución, la red, la publicación y la capacidad de diagnóstico.

Prueba	Procedimiento	Resultado esperado	Evidencia
Paquete	`dpkg -l apache2`	Paquete instalado	Salida de consola
Versión	`apache2ctl -v`	Versión identificada	Salida de consola
Servicio	`systemctl is-active apache2`	`active`	Salida de consola
Inicio automático	`systemctl is-enabled apache2`	`enabled`	Salida de consola
Puerto	`sudo ss -ltnp 'sport = :80'`	Apache escucha en TCP 80	Salida de consola
Configuración	`sudo apache2ctl configtest`	`Syntax OK`	Salida de consola
Acceso local	`curl -I http://localhost`	Respuesta HTTP satisfactoria	Cabeceras HTTP
Acceso remoto	Navegador desde otro equipo	Página de AulaDeploy	Captura con la URL
HTML	Solicitar `/`	Documento completo	Página mostrada
CSS	Solicitar `/assets/css/estilos.css`	Respuesta `200` y estilos aplicados	Cabeceras y captura
Recurso inexistente	Solicitar `/no-existe.html`	Respuesta `404`	Cabeceras y `access.log`
Registros	Consultar `access.log`	Peticiones y estados identificados	

Una prueba debe indicar qué se ha comprobado, qué resultado se esperaba, qué se obtuvo realmente y si es necesario aplicar alguna corrección.

Actividad práctica: primer servidor web de AulaDeploy

El plan de validación anterior se aplicará de forma completa en la práctica de AulaDeploy, en la que cada operación deberá quedar respaldada por una evidencia.

Situación

El equipo de AulaDeploy necesita un servidor web inicial que permita comprobar la conectividad y publicar información estática antes de desplegar la aplicación Java. La máquina virtual dispone de Ubuntu Server, pero todavía no se ha validado el servicio web.

Trabajo que debe realizarse

1. Registrar los datos del entorno.
2. Actualizar el índice de paquetes.
3. Instalar Apache HTTP Server.
4. Identificar la versión instalada.
5. Comprobar el estado y el inicio automático.
6. Verificar la escucha en el puerto 80.
7. Comprobar el acceso local.
8. Comprobar el acceso desde el equipo anfitrión.
9. Sustituir la página predeterminada por la página de AulaDeploy.
10. Aplicar propietarios y permisos adecuados.
11. Validar la configuración.
12. Generar una petición correcta y otra inexistente.
13. Localizar ambas peticiones en los registros.
14. Documentar el proceso.

Restricciones

- No utilizar `chmod 777`.
- No desactivar el cortafuegos como solución permanente.
- No modificar todavía el puerto.
- No crear sitios virtuales adicionales.
- No instalar PHP, Tomcat, bases de datos ni Docker.
- No ocultar errores mediante una reinstalación sin diagnóstico.

Resultado

El sitio debe ser accesible desde el equipo anfitrión mediante la dirección IP del servidor y debe mostrar correctamente el HTML y la hoja de estilo.

Documentación, evidencias y lista de comprobación

La práctica anterior no queda completa cuando el servidor comienza a responder. También deben conservarse pruebas suficientes para demostrar qué se ha configurado, cómo se ha validado el funcionamiento y qué incidencias se han resuelto. La memoria técnica y la lista de comprobación permiten organizar esas evidencias y verificar que no se ha omitido ninguna parte del procedimiento.

Estructura de la memoria técnica

1. Identificación del equipo y del sistema operativo.
2. Dirección del servidor y configuración de red utilizada.
3. Versión de Apache instalada.
4. Órdenes ejecutadas y finalidad de cada una.
5. Estado del servicio.
6. Puerto de escucha.
7. Estructura publicada.
8. Propietarios y permisos aplicados.
9. Resultado de la validación de sintaxis.
10. Prueba local.
11. Prueba desde otro equipo.
12. Peticiones identificadas en los registros.
13. Incidencias encontradas.
14. Soluciones aplicadas.
15. Resultado final.

Lista de comprobación

- ○ Apache está instalado.
- ○ La versión real está documentada.
- ○ El servicio está activo.
- ○ El inicio automático está habilitado.
- ○ Apache escucha en el puerto 80.
- ○ La configuración devuelve Syntax OK.
- ○ La prueba local funciona.
- ○ La prueba desde otro equipo funciona.
- ○ La página de AulaDeploy sustituye a la predeterminada.
- ○ El CSS se carga correctamente.
- ○ Los permisos no conceden escritura general.
- ○ Los accesos aparecen en el registro.
- ○ Se ha provocado e identificado un 404.
- ○ Las incidencias están documentadas.

Apache ya puede recibir peticiones HTTP y publicar contenido estático. También conocemos las ubicaciones básicas de configuración, contenido y registros.

El siguiente apartado ampliará esta instalación mediante módulos y sitios virtuales. Esa ampliación partirá del servidor funcional y documentado que acabamos de validar.

Referencias técnicas

Apache Software Foundation. *Apache HTTP Server Version 2.4 Documentation.*
https://httpd.apache.org/docs/2.4/

Apache Software Foundation. *Getting Started with the Apache HTTP Server.*
https://httpd.apache.org/docs/2.4/getting-started.html

Apache Software Foundation. *Log Files — Apache HTTP Server Version 2.4.*
https://httpd.apache.org/docs/2.4/logs.html

Apache Software Foundation. *apachectl — Apache HTTP Server Control Interface.*
https://httpd.apache.org/docs/2.4/programs/apachectl.html

Canonical. *How to install Apache2 — Ubuntu Server documentation.*
https://ubuntu.com/server/docs/how-to/web-services/install-apache2/

Canonical. *How to configure Apache2 settings — Ubuntu Server documentation.*
https://ubuntu.com/server/docs/how-to/web-services/configure-apache2-settings/

Canonical. *Firewall — Ubuntu Server documentation.* **https://ubuntu.com/server/docs/firewalls/**

Ubuntu Manpage Repository. *apache2ctl — Apache HTTP server control interface.*
https://manpages.ubuntu.com/manpages/noble/man8/apache2ctl.8.html

---

Ubuntu Manpage Repository. *apache2 — Apache Hypertext Transfer Protocol Server.*
https://manpages.ubuntu.com/manpages/noble/man8/apache2.8.html